

DuoQ: A DSP Utilization-aware and Outlier-free Quantization for FPGA-based LLMs Acceleration

Zhuoquan Yu*, Huidong Ji*, Yue Cao, Junfu Wu, Xiaozhe Yan, Lirong Zheng[†], and Zhuo Zou[†]

State Key Laboratory of Integrated Chips and Systems, Fudan University, Shanghai, China

Email: {zqyu21, hdji21, caoyue23, junfuwu23, xzyan23}@m.fudan.edu.cn, {lrzheng, zhuo}@fudan.edu.cn

Abstract—Quantization enables efficient deployment of large language models (LLMs) on FPGAs, but its presence of outliers affects the accuracy of the quantized model. Existing methods mainly deal with outliers through channel-wise or token-wise isolation and encoding, which leads to expensive dynamic quantization. To address this problem, we introduce DuoQ, an FPGA-oriented algorithm-hardware co-design framework. DuoQ effectively eliminates outliers through learnable equivalent transformations and low-semantic token awareness in the quantization scheme part, facilitating per-tensor quantization with 4-bits. We co-design the quantization algorithm and hardware architecture. Specifically, DuoQ accelerates end-to-end LLM through a novel DSP-aware PE unit design and encoder design. In addition, two types of post-processing units assist in the realization of nonlinear functions and dynamic token awareness. Experimental results show that compared with platforms with different architectures, DuoQ’s computational efficiency and energy efficiency are improved by up to 8.8× and 23.45×. In addition, DuoQ has achieved accuracy improvements compared to other outlier-aware software and hardware works.

Index Terms—FPGA, W4A4 quantization, LLMs, DSP utilization, algorithm-hardware co-design

I. INTRODUCTION

Transformer-based large language models (LLMs) are gaining attention for their excellent performance in natural language processing (NLP) scenarios such as text understanding and generation. However, the construction and processing of models involve a large number of parameters and operations, which also pose challenges [1], [2]. For example, the GPT3-175B (FP16) requires 326 GB of memory, and the amount of computation required to complete an inference is approximately 660 TOPS.

Field-programmable gate array (FPGA) is a potential hardware platform with flexible resource scheduling and tailored architecture design to accelerate LLM inference and explore the benefits of optimization algorithms [3]–[5]. However, its computational capabilities remain largely underutilized. Efficiently utilizing DSPs, core FPGA units optimized for arithmetic tasks like multiplication and accumulation, can support more parallel operations, reducing computational load and boosting throughput in LLM tasks.

Quantizing those parameters to a low bit-width is a powerful technique to achieve optimal utilization for better computational throughput [6]. It can also greatly reduce memory footprint and bandwidth pressure. However, quantizing activations

This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB4704101. (*Equal contribution, [†]Corresponding author.)

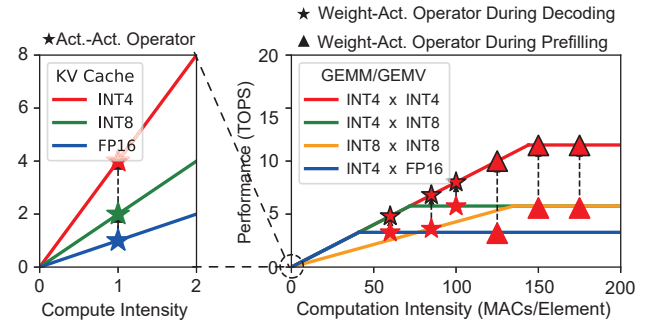


Fig. 1. Roofline model of DSP-driven FPGA platform.

to low bit-widths, like 4-bit, in LLM operations can cause outliers, significantly degrading model accuracy. Previous work has proposed many types of methods to address this problem [7]–[11], such as channel-wise scaling, mixed-precision quantization with dedicated hardware support, Hadamard rotation, and channel-wise assembly. These methods all heuristically mark outliers and dynamically quantize the channels and tokens where the outliers are located, and cannot eliminate the impact of outliers. This paper aims to eliminate outliers at the tensor level while minimizing hardware overhead. We leverage the observation that pervasive channel outliers manifest across blocks and layers, as shown in Fig. 2(a) and Fig. 2(b), and that tokens with low semantic specificity exhibit significant outlier behavior.

We introduce DuoQ, a complete mapping and outliers-free algorithm-hardware co-design framework for LLM implementation on FPGAs, which ensures a balance between accuracy and hardware efficiency. We first reveal the potential optimization space of current FPGA accelerators for LLM, and analyze the distribution of outliers and corresponding optimization schemes. Then, to maximize the elimination of the impact of outliers, we propose a W4A4KV4 (4-bit weight, 4-bit activation, and 4-bit KV cache) per-tensor quantization framework. We build and implement an FPGA-based accelerator architecture compatible with the quantization framework. DuoQ achieves efficient computation while ensuring accuracy through comprehensive algorithm and hardware collaboration. The main contributions of this paper are listed as follows:

- We propose DuoQ, an FPGA-oriented algorithm-hardware co-design framework, as shown in Fig. 3, which hosts an outlier-free quantization scheme and a corresponding FPGA architecture design.
- We introduce an outlier-free quantization scheme jointly optimized by the equivalent transformations and the low-

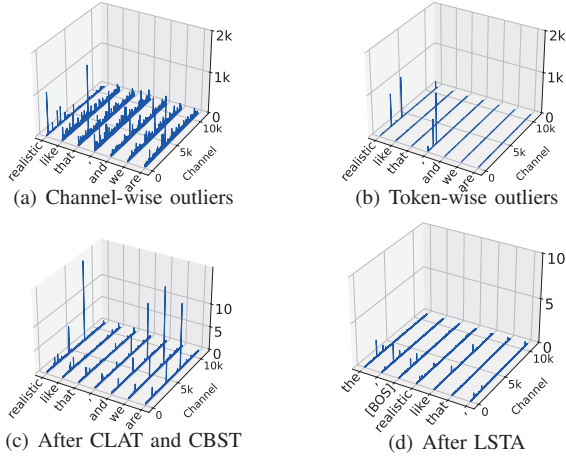


Fig. 2. Statistical chart of LLaMA2-7B intermediate activation values.

semantic-token awareness methods to ensure accuracy.

- A co-designed FPGA-based architecture is built to improve the DSP's resource utilization and computation efficiency.
- We implement the FPGA-based architecture from the framework. The experimental results in LLaMA-7B and LLaMA-13B show that DuoQ reduces WikiText2 perplexity by 5.56 and 2.31, respectively, compared to other state-of-the-art (SOTA) outlier-aware hardware works. Moreover, compared to platforms with different architectures, DuoQ improves computational efficiency and energy efficiency by a maximum of $8.8\times$ and $23.45\times$.

II. BACKGROUND AND MOTIVATION

A. Related Accelerators for LLMs

Recently, transformer-based LLM accelerators have been proposed, but they face three key limitations. a) [3], [12]–[14] mainly focus on accelerating the attention mechanism by pruning or quantization. They do not implement end-to-end LLM deployment and ignore the equally important linear layer computation with computational bottlenecks. b) Although FPGA-based accelerators [15]–[17] provide comprehensive acceleration, they often overlook software-driven algorithmic enhancements for performance. For example, low-bit quantization, as shown in Fig. 1, INT4-INT4 quantization compared to other precisions has obtained different levels of performance improvements in the prefill and decode stages of the model. c) [6], [18], [19] consider advanced quantization algorithms that include outliers and are co-designed with the hardware. However, their methods for encoding and isolating outliers are limited by fixed quantization strategies and input data and are not optimal.

Moreover, FPGA-based accelerators such as [16] and [18] do not fully utilize the important DSP resources, and the optimization potential has not been fully explored. In FPGAs, DSP-based MAC operations have higher resource utilization than LUTs, with a computational efficiency of about 90 times that of LUTs, as shown in [20] and [21]. Therefore, it is important to make efficient use of DSP resources. Current DSPs support INT8 multiplication, however, the extended support for INT4 is limited by the finite bit length and cannot

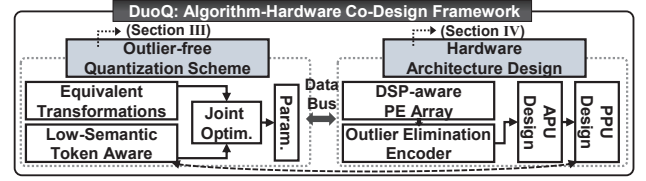


Fig. 3. The proposed algorithm-hardware co-design framework.

describe the complete information. [16], [18], [22] claim the INT4 quantization support, but the W4A8-based mixed precision method proves no computation efficiency improvement, compared to the INT8-based DSP packing. [23] attempts to deploy a converter with INT4 multiplication on an FPGA, but since it cannot support signed operands as inputs, the INT4 implementation is limited to the post-nonlinearity layer. [24] utilizes the ability to run six multiplications in parallel, but still requires significant external hardware resources to help recover the required information and accuracy.

B. W4A4KV4 Quantization and Outliers Analysis

Fig. 2 shows the activation distribution of a representative linear layer in LLaMA2-7B. It contains many outliers as mentioned in the works [7], [8], as shown in Fig. 2(a), prevalent across channels and manifest across blocks and layers. More accurate encoding of these activations and outlier isolation can alleviate this problem, as shown in [6], [14], [19], but at the cost of mixed-precision methods and overheads such as reordering. More importantly, this approach cannot adapt to diverse input sequences, as the token-wise anomalies are input-dependent. Other works try to balance the weight and activation anomalies by adjusting the per-channel scaling of the diagonal elements in the matrix multiplication view. Recent works such as hadamard transformation [10], [11] have considered the features across layers and individual features, but the transformed activations are not smooth enough. They also do not take into account tokens that cause a large number of outliers. In addition, consistent with the observations in [25], the starting token and the first special token in Fig. 2(b) will exhibit large activation values. In this paper, they are collectively referred to as low-semantic tokens. It should be noted that the distribution law is not consistent among different models, so we will use the joint optimization method proposed in Section III-C for targeted optimization.

C. An Algorithm-Hardware Co-Design for INT4 LLM's Highly Efficient Implementation on FPGA

In response to these challenges concerning the quantization and hardware design, we propose the algorithm-hardware co-design framework, as displayed in Fig. 3. Concerning the quantization scheme, equivalent transformations across layers and blocks are designed to optimize weight and activation distributions, effectively mitigating outliers. The Low-semantic-awareness attention method is used to flatten the token-wise outliers, shown in Fig. 2(c), to reduce the accuracy loss, exhibited in Fig. 2(d).

An FPGA-based architecture is built to enable the quantized LLM deployment with improved computation and energy efficiency following the listed optimizations: 1) A novel

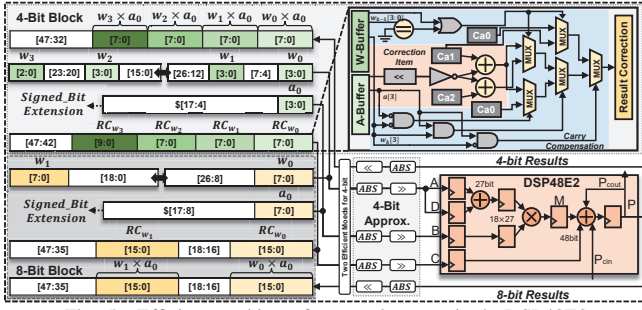


Fig. 5. Efficient packing of operands on a single DSP48E2.

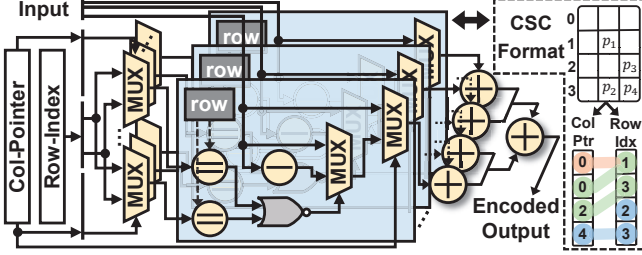


Fig. 6. Encoding the intermediate activation to eliminate outliers.

where $\pi = \{\mathbf{P}, \bar{\mathbf{s}}\}$ and $\|\cdot\|_F$ is the Frobenius norm. Given the quantization parameter θ and the token τ embedded in the model KV caches, $\mathcal{F}(\cdot)$ and $\hat{\mathcal{F}}_{\theta, \tau}(\cdot)$ denote the outputs before and after quantization of the transformer layer under the input tensor $\mathbf{x}$, respectively. $\mathcal{T}(\mathbf{x})$ is the equivalent transformation in Section III-A. It is therefore solved for

$$\min_{\theta, \pi, \tau} \mathcal{L}(\theta, \pi, \tau). \quad (4)$$

Similar to the case of [26], this formulation is analogous to the k-means clustering process. Consider θ and τ as the cluster centers, which define the mapping before and after quantization. π is then considered as the mapping from the sample points to the cluster centers. This problem is then solved using an alternating optimization strategy. First, π is fixed and a subproblem about θ and τ is solved, i.e., $\theta^t, \tau^t \leftarrow \arg \min_{\theta, \tau} \mathcal{L}(\theta, \pi^{t-1}, \tau)$, where t is the alternating index. They are optimized by stochastic gradient descent. Since they are fixed, the gradients are not backpropagated to π^{t-1} . Next, fixing θ and τ , the subproblem about π is solved, i.e., $\pi^t \leftarrow \arg \min_{\pi} \mathcal{L}(\theta^t, \pi, \tau^t)$. The key to this optimization is the optimization of $\mathcal{T}(\mathbf{x})$, i.e.,

$$\min_{\mathbf{P}} \|\mathbf{xP} - Q(\mathbf{xP})\|_F^2. \quad (5)$$

This is an equilibrium Procrustes problem subject to the constraint that $\mathbf{P} \in \mathcal{S}$ is an orthogonal stiefel manifold. We can obtain an explicit solution by singular value decomposition of the matrix $\mathbf{x}^T Q(\mathbf{xP})$. Then, we perform an alternating approximation step, substitute the approximate π^t into the subproblems of θ^t and τ^t , and continue the alternating solution until convergence.

IV. HARDWARE ARCHITECTURE DESIGN

A. Overall Architecture

Fig. 4(b) shows the overall architecture of DuoQ. It adopts a five-stage pipeline structure to ensure hardware utilization: **1) Preloading:** Due to the large read latency of HBM, the

weight blocks are preloaded into 32 64-depth $\times$ 16B FIFOs and converted by the bitwidth convactor. **2) Loading off-chip data:** After the control unit has propagated the control signals to the bitwidth converter, the weights and input data are read from the FIFOs into the buffer. The buffer consists of a BRAM array distributed across the input buffers of each token feature processing element (TFPE) and a global buffer. **3) MAC operations:** The data corresponding to C_{in} and C_{out} flows through the PE array in each TFPE and is multiplied and accumulated. In particular, the PE units are designed to efficiently calculate signed 4-bit numbers from the input via DSP-packing. **4) Post-processing operations:** The auxiliary processing unit (APU) performs non-linear operations on the intermediate results. For the encoder, it ensures that the output values are returned to storage and the KV caches are accurately quantized without outliers. **5) Writing data back to off-chip memory:** Finally, the output buffer is also written back to HBM via 32 FIFOs. The qkv-fetcher before the output buffer writes the address of the corresponding vector back with the result of the token importance of the post processing unit (PPU), to combine Section III-B.

B. DSP-aware and Reconfigurable PE Unit Design

As shown in Fig. 5, the DSP's A and D input interfaces support $K \in \{2, 4, 8\}$ w_k packings in different calculation modes. They share the same activation a_0 and input to B port. The label of each w_k is denoted as $k \in \mathcal{K} = \{0, 1, \dots, K\}$. Then, the n -th bit of w_k is denoted as $n \in \mathcal{N} = \{0, 1, \dots, N\}$, where $N \in \{4, 8\}$ denotes the support of the maximum bit width of our design. Therefore, w_k of N bits is represented as $w_k[N-1:0] = \{w_k[N-1], \dots, w_k[n], \dots, w_k[0]\}$, where $[\cdot]$ is used to indicate the index.

1) Activation Shared Signed 8-bit Packed Calculation:

When $N = 8$ and $K = 2$, since the sign bit of $w_0(k = 0)$ is not on the most significant bit (MSB), the actual result obtained is not the target $a_0 \cdot w_0$, but the unsigned $a_0 \cdot \hat{w}_0$ generated by $a_0 \cdot w_0$. $\hat{w}_0$ can be expressed as $\hat{w}_0 = \sum_{n=0}^{N-2} w_0[n] \cdot 2^n + w_0[N-1] \cdot 2^{N-1}$. Then, we transform the expression of $\hat{w}_0$ to

$$\hat{w}_0 = w_0 + w_0[N-1] \cdot 2^N. \quad (6)$$

From this, we can get $a_0 \cdot \hat{w}_0 = a_0 \cdot w_0 + a_0 \cdot w_0[N-1] \cdot 2^N$. In this case, the result after multiplication, $a_0 \cdot \hat{w}_0$, needs to be corrected by subtracting the correction term $Ct_{w_0} = a_0 \cdot w_0[N-1] \cdot 2^N$ to obtain the correct result. And when a_0 is signed, the result $a_0 \cdot w_1$ needs to be corrected by adding the correction term $Ct_{w_1} = 1$.

2) Improved Signed 4-bit Packed calculation: When $N = 4$ and $K = 4$, the DSP48E2 specifications do not support packing the MSB of w_{K-1} . Instead of the desired result, a_0 and w_{K-1} are fed into the DSP calculation, resulting in $a_0 \cdot \tilde{w}_{K-1}$ with the 3-bit signed number $\tilde{w}_{K-1}[2:0] = \{w[2], w[1], w[0]\}$. From $\tilde{w}_{K-1} = \sum_{n=0}^{N-3} w_{K-1}[n] \cdot 2^n - w_{K-1}[2] \cdot 2^2$, by splitting we get

$$\tilde{w}_{K-1} = w_{K-1} - w_{K-1}[2] \cdot 2^3 + w_{K-1}[3] \cdot 2^3. \quad (7)$$

Therefore, for $a_0 \cdot \tilde{w}_{K-1}$, it can be re-expressed as $a_0 \cdot \tilde{w}_{K-1} = a_0 \cdot w_{K-1} - a_0 \cdot (w_{K-1}[2] - w_{K-1}[3]) \cdot 2^3$. Similarly, we get

Algorithm 1 Integer-Only PWL, $k_i x + t_i (s_i \leq x \leq s_{i+1})$.

Input: x_q, ε : quantized input and scaling factor.

Output: y_q, ε_y : quantized output and scaling factor.

```

1: Function I-PWL( $x_q, \varepsilon$ )  $\triangleright x_q \varepsilon = x$ 
2:    $s_q \leftarrow \lfloor s/\varepsilon \rfloor, k_q \leftarrow \lfloor k/\varepsilon \rfloor$ 
3:    $t_q \leftarrow \lfloor t/\varepsilon^2 \rfloor, \varepsilon_y \leftarrow \varepsilon^2, y_q = k_q x_q + t_q$ 
4:   return  $y_q, \varepsilon_y$   $\triangleright y_q \varepsilon_y \approx kx + t$ 
5: end Function

```

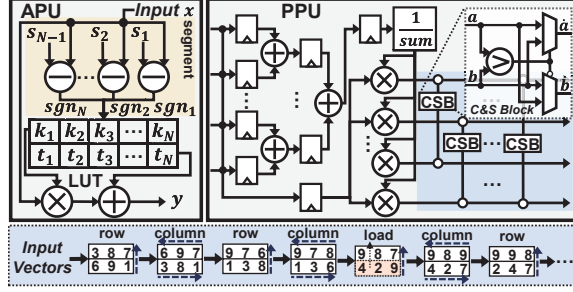


Fig. 7. Hardware microarchitecture of APU and PPU.

the correction term for w_{K-1} as $Ct_{w_{T-1}} = a_0 \cdot (w_{K-1}[2] - w_{K-1}[3]) \cdot 2^3$, which is implemented using only shifters, adders and bitwise AND units. The result of $a_0 \cdot \hat{w}_k$ obtained by packing the low-order bits of $\{w_k \mid k = 0, 1, 2\}$ in the input A, D is also corrected using the method in (6). In addition, signed-signed multiplications will generate carry between adjacent results after packing. Therefore, in addition to w_0 being unaffected, $\{w_k \mid k = 1, 2, 3\}$ requires additional carry compensation ($Ca0$ to $Ca2$) in addition to the respective correction terms. Fig. 5 shows the compensation terms constructed for w_1 and w_2 , and w_3 is constructed similarly.

In addition, based on the advantage of the designed structure that expands the calculation using the common operand a , we have designed an approximation technique that can be applied to signed-unsigned calculations where most probabilities are close to 0, such as after softmax, with almost no loss of precision. Specifically, on the basis of the above complete 4-bit packing, an active bit scanner (ABS) and a shifter are added to the DSP input. The number for each port is first entered into the ABS by accumulation, and then for positive numbers, the ABS is indicated by detecting the first active bit equal to 1 (0 for negative), and then shifting right to 4 bits. Finally, the output is restored by shifting left. Thus, each port supports the packing of up to 16 unsigned numbers.

C. Encoder Design for Outlier Elimination

For the equivalent transformation in Section III-A, we design an encoder to be applied in the intermediate activation and eliminate outliers. The properties of $\mathbf{P}_h, h \in \{q, k, dn\}$ are similar to sparsity, and the distribution of $p = -1$ in the matrix is similar to the distribution of non-zero values in sparsity. Therefore, we consider formatting and storing $\mathbf{P}$ in a lookup table first by using the compressed sparse column (CSC) format [27]. Then the encoder in Fig. 6 is used to encode the input activations. Benefiting from this, the proposed encoder can be configured to work in sparse mode, which is applied to the sparse values in $\mathbf{A}_b \mathbf{W}_b^T$ in addition to the $\mathbf{P}$ transformation. In the encoder design, input activation first selects the column containing p via Col_Ptr , and then

TABLE I

PERPLEXITY COMPARISON ON WIKITEXT WITH W4A4 QUANTIZATION

Model	L-7B	L-13B	L-30B	L-65B	O-6.7B	O-13B	O-30B	O-66B
FP16	5.68	5.09	4.1	3.53	10.86	10.12	9.56	9.34
OmniQuant [7]	11.26	10.87	10.33	9.17	12.24	11.65	10.6	10.29
Olive [19]	144.75	42.24	36.55	1.4e7	107.15	416.57	334.7	4058.83
Oltron [6]	11.67	8.2	6.68	5.82	11.99	11.61	10.64	10.5
HotaQ [18]	10.72	9.07	7.39	6.02	13.84	13.9	11.02	11.29
DuoQ	6.11	5.89	5.14	4.23	11.31	11.60	10.44	10.21

*L: LLaMA Model, *O: OPT Model, Perplexity: the lower the better

uses Row_Idx to perform complement and accumulation operations for each set of activations. In addition, when the constructed $\mathbf{P}$ does not satisfy the size of 2^n , CLAT uses the Kronecker product to decompose it.

D. Auxiliary Processing Unit

The APU is designed to be used with non-linear functions to support the softmax function, normalization function and activation functions such as SiLU in LLMs. To implement them efficiently without loss of precision, the APU uses a piecewise linear (PWL) approximation. To efficiently implement this approximation, we use a neural network approach to approximation, as in [28]. The parameters of each segment $k_i x + t_i$ if $(s_i < x < s_{i+1})$ are implemented using LUTs. The APU makes two important improvements. The first is to construct an integer-only PWL function as shown in Algorithm 1 to improve execution efficiency. The second is to achieve the best approximation by fine-tuning a piece of data. The structure of the APU is shown on the left side of Fig. 7. It can achieve an MAE of 2.07×10^{-3} for the sigmoid function in experiments, which is better than previous work.

E. Post Processing Unit

The PPU first performs the post-processing operations in Section IV-D. For example, in the case of softmax, the input values after the exponential transformation are summed using an addition tree and the reciprocal is calculated, and then multiplied with the input one by one to obtain a normalized probability distribution. They are then passed to our carefully designed real-time window-based sorter (RTWS) for maximum value set filtering. Due to the pipelined architecture described in Section IV-A and the on-chip memory constraints, the sorter cannot receive all the data in one clock cycle. and requires a lot of on-chip storage when processing large-scale computations of LLM, so a similar bitonic sorting network is not feasible in our scenario. In RTWS, it is also composed of basic modules, the compare and swap block (CSB). Its window comparison process is shown in the lower part of Fig. 7. This effectively saves on-chip storage resources.

V. EVALUATION

A. Methodology

For the accuracy evaluation of the quantization, our method is evaluated on LLaMA [2] and OPT [1] families with W4A4. We evaluate the perplexity on WikiText [29] and accuracy on four datasets [30]–[33]. DuoQ is compared with outlier-aware hardware methods [6], [7], [18], [19], as shown in Table I. We also compared it with the SOTA works of the algorithm design [8]–[11], as shown in Table II. And 128 random sampled sequences from [29] are used for calibration.

TABLE II
ZERO-SHOT ACCURACY OF QUANTIZED LLAMA MODELS

Model	Method	Accuracy↑					
		piga	hella	wino	arc-e	arc-c	Avg.
LLaMA2-7B	fp16	79.11	76.00	69.53	74.33	46.25	69.04
	SmoothQuant [9]	51.90	27.05	48.38	25.63	26.37	35.87
	Atom [8]	75.14	69.37	69.37	52.99	38.40	59.73
	QuaRot [10]	76.17	72.3	64.40	68.06	41.47	64.48
	SpinQuant [11]	76.12	73.15	66.54	69.32	41.64	65.35
	DuoQ	76.87	73.12	69.38	71.11	42.23	66.54
LLaMA2-13B	fp16	80.52	79.37	72.22	77.48	49.06	71.73
	SmoothQuant	53.81	30.70	51.07	33.00	24.49	38.61
	Atom	76.50	73.84	67.40	57.49	42.32	63.51
	QuaRot	77.48	75.28	67.88	72.35	45.65	67.73
	SpinQuant	78.56	77.01	67.88	75.97	46.76	69.24
	DuoQ	79.33	76.91	71.13	74.88	47.11	69.87
LLaMA2-70B	fp16	82.54	84.31	79.48	80.30	56.91	76.71
	Atom	79.92	79.06	74.27	58.25	46.08	67.52
	QuaRot	82.43	81.82	76.24	80.43	56.23	75.43
	SpinQuant	81.61	82.36	75.85	79.17	56.31	75.05
	DuoQ	81.91	83.01	77.87	79.33	56.37	75.69
	fp16	80.69	79.17	72.61	77.69	53.41	72.71
LLaMA3-8B	SmoothQuant	51.09	28.90	50.36	28.96	23.55	36.57
	QuaRot	75.63	72.38	65.98	67.30	44.45	65.15
	SpinQuant	77.04	69.22	74.83	74.07	45.99	68.23
	DuoQ	79.12	74.31	73.29	77.91	51.48	71.22
	fp16	84.49	84.90	80.51	85.90	64.33	80.03
	QuaRot	77.37	76.75	68.51	72.31	47.01	68.39
LLaMA3-70B	SpinQuant	80.80	77.30	56.00	80.90	76.40	74.28
	DuoQ	84.04	83.77	76.77	80.92	70.15	79.13

For the hardware design part, we simulated and synthesized the RTL-based architecture on AMD Xilinx Vivado 2020.1 and implemented it on the Xilinx Alveo U280. Furthermore, Xilinx Power Estimator and switching activity interchange format file (SAIF) are jointly utilized to estimate the runtime power consumption of the design. SOTA-related FPGA accelerators [15], [16], [18] and other platforms are used as the baseline.

B. Accuracy Evaluation

We first report a comparison of DuoQ with other outlier-aware hardware on perplexity. As shown in Table I, DuoQ outperforms existing methods on various models. Olive and Oltron’s methods for encoding outliers make it difficult to accurately identify all, which does not best preserve the model’s accuracy. Omniquant smoothes the activation distribution but does not perform well on large models. HotaQ, while taking into account the perception of attention activation, has limited smoothing of the overall activation.

We conduct another software evaluation of the zero-shot accuracy of quantized LLMs in Table II. For the LLaMA2 (7B-70B) model, our model shows the highest accuracy in almost all datasets, with a maximum accuracy loss of only 1.86% compared to FP16. For the latest LLaMA3 (8B-70B), we have achieved an accuracy improvement of 2.99% and 4.85% compared to the SOTA working SpinQuant.

C. Analysis of PE Unit

Table III summarizes the DSP-packing methods considering different performance metrics. In terms of the supported data width, the DSP-packing method of our PE supports both the INT4 and INT8 operands, also the design is capable of multiplying both signed operands, crucial for the application’s correctness. For the resource utilization with the implementation, our method claims the maximum resource conservation.

D. Analysis of System Performance

In this work, the proposed hardware architecture design with the W4A4KV4 high processing capabilities is evaluated

TABLE III
COMPARISON OF DIFFERENT DSP-PACKING METHODS

Method	Data Type	Signed Mult. Support	Utilization per Multiplication		
			DSP	LUT	FF
WP486 [34]	INT8	No	0.5	0	0
DoubleMAC [35]	INT8	Yes	0.5	11	12
ParallelDot [36]	INT8	Yes	0.125	24.63	25.13
WP521 [37]	INT4	Partial	0.25	0	0
DSP_Packing [24]	INT4	Partial	0.18	4.5	5.33
TransFRU [23]	INT8/INT4	Yes/Partial	0.5/0.125	7/6.25	8.5/5
Ours	INT8/INT4/INT4	Yes/Yes/Partial	0.5/0.25/0.02	3.5/5.25/1.52	6/7.5/5

TABLE IV
FPGA IMPLEMENTATION RESULTS

Device	AMD Xilinx Alveo U280(16nm)			
Frequency	150 MHz			
Memory	8(DDR) & 32(HBM) GB			
Resource	LUT	FF	DSP	BRAM
Total	484k (37.1%)	670k (18.5%)	8192 (90.7%)	668 (33.1%)

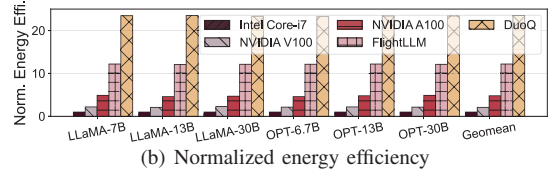
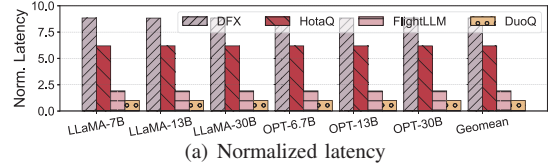


Fig. 8. Comparison of platforms built of different architectures.

with the available quantized INT4 LLM models evaluated in Section V-B, and the sequence length of 2048 is considered as the standard length of input and output tokens.

Table IV displays the implementation result of our design on Xilinx Alveo U280 operating at the frequency of 150 MHz. The runtime latency and the energy efficiency are reported in Fig. 8(a). Compared with other SOTA FPFA-based LLM accelerators [15], [16], [18], our work observes a minimum improvement of $1.9\times$ and a maximum improvement of $8.8\times$ in terms of the normalized runtime latency.

In consideration of our work’s improvement of energy efficiency over other platforms. The power consumption processing of different LLM works under different architectures are recorded and analyzed in Fig. 8(b). A minimum of $1.9\times$ and a maximum of $23.45\times$ improvements are found compared to the FlightLLM FPGA and Intel Core-i7 CPU, respectively.

VI. CONCLUSION

In this work, we propose DuoQ, an FPGA-oriented algorithm-hardware co-design framework to enable high DSP utilization and the free outlier during LLM inference acceleration. In the algorithm optimization part, it introduces a hardware-friendly INT4 quantization to save the hardware cost while maintaining accuracy. In the hardware design part, an FPGA-based accelerator is built to enable the DSP’s support for INT4 efficient multiplication. DuoQ achieves satisfactory accuracy among different datasets through the quantization scheme, averaging 66.54%, and the corresponding implemented accelerator improves the computation efficiency with a maximum of $8.8\times$ and $23.45\times$ for the energy efficiency.

REFERENCES

- [1] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, *et al.*, “Opt: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [3] T. J. Ham, S. J. Jung, S. Kim, Y. H. Oh, Y. Park, Y. Song, J.-H. Park, S. Lee, K. Park, J. W. Lee, and D.-K. Jeong, “A³: Accelerating attention mechanisms in neural networks with approximation,” in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 328–341, 2020.
- [4] M. Muller, A. Tyshka, M. Theisen, and D. Hanna, “Co-design of a tinyllm using programmable logic and software on an fpga,” in *2024 IEEE 67th International Midwest Symposium on Circuits and Systems (MWSCAS)*, pp. 253–257, 2024.
- [5] P. Qi, Y. Song, H. Peng, S. Huang, Q. Zhuge, and E. H.-M. Sha, “Accommodating transformer onto fpga: Coupling the balanced model compression and fpga-implementation optimization,” in *Proceedings of the 2021 Great Lakes Symposium on VLSI, GLSVLSI ’21*, p. 163168, 2021.
- [6] C. Xue, C. Zhang, X. Jiang, Z. Gao, Y. Lin, and G. Sun, “Oltron: Algorithm-hardware co-design for outlier-aware quantization of llms with inter-/intra-layer adaptation,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference, DAC ’24*, 2024.
- [7] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, “Omniquant: Omnidirectionally calibrated quantization for large language models,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [8] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, “Atom: Low-bit quantization for efficient and accurate llm serving,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 196–209, 2024.
- [9] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: accurate and efficient post-training quantization for large language models,” in *Proceedings of the 40th International Conference on Machine Learning, ICML’23*, 2023.
- [10] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefer, and J. Hensman, “Quarot: Outlier-free 4-bit inference in rotated llms,” *arXiv preprint arXiv:2404.00456*, 2024.
- [11] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort, “Spinquant-llm quantization with learned rotations,” *arXiv preprint arXiv:2405.16406*, 2024.
- [12] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, “Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 811–824, 2020.
- [13] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, “Elsa: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pp. 692–705, 2021.
- [14] W. Li, A. Hu, N. Xu, and G. He, “Quantization and hardware architecture co-design for matrix-vector multiplications of large language models,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 6, pp. 2858–2871, 2024.
- [15] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, “Dfx: A low-latency multi-fpga appliance for accelerating transformer-based text generation,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 616–630, 2022.
- [16] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang, Y. Dai, J. Li, Z. Wang, R. Zhang, K. Wen, X. Ning, and Y. Wang, “Flightllm: Efficient large language model inference with a complete mapping flow on fpgas,” in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays, FPGA ’24*, pp. 223–234, 2024.
- [17] H. Chen, J. Zhang, Y. Du, S. Xiang, Z. Yue, N. Zhang, Y. Cai, and Z. Zhang, “Understanding the potential of fpga-based spatial acceleration for large language model inference,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, no. 1, pp. 1–29, 2024.
- [18] X. Shen, Z. Han, L. Lu, Z. Kong, P. Dong, Z. Li, Y. Xie, C. Wu, M. Leiser, P. Zhao, X. Lin, and Y. Wang, “Hotaq: Hardware oriented token adaptive quantization for large language models,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–1, 2024.
- [19] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, “Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA ’23*, 2023.
- [20] S. Ullah, S. Rehman, M. Shafique, and A. Kumar, “High-performance accurate and approximate multipliers for fpga-based hardware accelerators,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 2, pp. 211–224, 2022.
- [21] A. C. Mert, H. Azgin, E. Kalali, and I. Hamzaoglu, “Efficient multiple constant multiplication using dsp blocks in fpga,” in *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*, pp. 331–3313, 2018.
- [22] G. Yang, Y. Xie, Z. J. Xue, S.-E. Chang, Y. Li, P. Dong, J. Lei, W. Xie, Y. Wang, X. Lin, and Z. Fang, “Sda: Low-bit stable diffusion acceleration on edge fpgas,” in *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*, pp. 264–273, 2024.
- [23] H. Wang, Y. Bai, J. Yu, and K. Wang, “Transfru: Efficient deployment of transformers on fpga with full resource utilization,” in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 521–526, 2024.
- [24] J. Sommer, M. A. zkan, O. Kesocze, and J. Teich, “Dsp-packing: Squeezing low-precision arithmetic into fpga dsp blocks,” in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*, pp. 160–166, 2022.
- [25] M. Sun, X. Chen, J. Z. Kolter, and Z. Liu, “Massive activations in large language models,” *arXiv preprint arXiv:2402.17762*, 2024.
- [26] X. Chen and K. He, “Exploring simple siamese representation learning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 15750–15758, 2021.
- [27] V. IsaacChassande, A. Evans, Y. Durand, and F. Rousseau, “Dedicated hardware accelerators for processing of sparse matrices and vectors: A survey,” *ACM Trans. Archit. Code Optim.*, vol. 21, Feb. 2024.
- [28] J. Yu, J. Park, S. Park, M. Kim, S. Lee, D. H. Lee, and J. Choi, “Nn-lut: neural approximation of non-linear operations for efficient transformer inference,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference, DAC ’22*, p. 577582, 2022.
- [29] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [30] Y. Bisk, R. Zellers, J. Gao, Y. Choi, *et al.*, “Piqa: Reasoning about physical commonsense in natural language,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, pp. 7432–7439, 2020.
- [31] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” *arXiv preprint arXiv:1803.05457*, 2018.
- [32] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, “Hel-laswag: Can a machine really finish your sentence?,” *arXiv preprint arXiv:1905.07830*, 2019.
- [33] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, “Winogrande: An adversarial winograd schema challenge at scale,” *Communications of the ACM*, vol. 64, no. 9, pp. 99–106, 2021.
- [34] Y. Fu, E. Wu, A. Sirasao, S. Attia, K. Khan, and R. Wittig, “Deep learning with int8 optimization on xilinx devices white paper (wp486),” 2024.
- [35] D. Nguyen, D. Kim, and J. Lee, “Double mac: doubling the performance of convolutional neural networks on modern fpgas,” in *Proceedings of the Conference on Design, Automation & Test in Europe, DATE ’17*, p. 890893, 2017.
- [36] M. Vstias, R. P. Duarte, J. T. de Sousa, and H. Neto, “Parallel dot-products for deep learning on fpga,” in *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*, pp. 1–4, 2017.
- [37] T. Han, T. Zhang, D. Li, G. Liu, L. Tian, D. Xie, and Y. S. Shan, “Convolutional neural network with int4 optimization on xilinx devices (wp521),” 2024.